

Military Asset Management System

Project Documentation & Technical Design

Version: Initial Framework **Date:** September 2026

1. Project Overview

The Military Asset Management System is a secure web application for managing the movement, assignment, expenditure and inventory of critical assets across multiple bases. The system is designed around accountability and traceability: authorized users can record purchases, transfer assets between bases, assign serialized assets to personnel, record expenditures, and review inventory movement through a dashboard.

Core capabilities

- Dashboard metrics for Opening Balance, Closing Balance, Net Movement, Assigned and Expended quantities.
- Filtering by date, base and equipment type.
- Purchase recording and purchase history.
- Inter-base transfer creation and transfer history.
- Asset assignment and expenditure tracking.
- Role-Based Access Control (RBAC) for Admin, Base Commander and Logistics Officer.
- Audit logging of important business transactions.

Assumptions

- Each user has one application role; a Base Commander is associated with an assigned base.
- Serialized equipment such as vehicles or individual weapons can be tracked as individual assets.
- Consumables such as ammunition are tracked by quantity rather than individual serial number.
- Transfers are treated as inventory movements between bases and are validated before being committed.
- Assignments reduce available inventory but do not by themselves represent permanent expenditure.

Limitations / scope

- This is an asset-accountability and logistics prototype, not an operational command-and-control system.
- Advanced enterprise features such as SSO, hardware-backed identity, high-availability database clusters and disaster-recovery orchestration are outside the initial scope.
- Production deployment should add organization-specific security controls, monitoring, backup policies and compliance review.

2. Tech Stack & Architecture

Frontend: React + Vite, with responsive UI components. React was selected for interactive dashboards, forms, filters, tables and role-specific views.

Backend: Python + FastAPI. FastAPI provides RESTful APIs, request validation through Pydantic, dependency injection, automatic OpenAPI documentation and a clean structure for authentication/RBAC middleware.

Database: MySQL, hosted as a managed cloud database. A relational database is appropriate because users, bases, assets, purchases, transfers and audit records have strong relationships and require transactional consistency.

ORM / DB access: SQLAlchemy with a MySQL driver. Database credentials are supplied through environment variables rather than hard-coded in source code.

Authentication: JWT-based authentication, with authorization checks performed by the backend.

Architecture

React Frontend → HTTPS REST API → FastAPI → Authentication/RBAC → Business Services → SQLAlchemy → Cloud MySQL

The frontend is responsible for presentation and user interaction. The backend is the security boundary: it validates authentication, role permissions and base-level access before executing business operations. The database stores persistent transactional and audit data.

3. Data Models / Schema

The core relational model contains the following entities:

Table	Purpose	Key relationships
users	Application users, roles and assigned base	FK → bases
bases	Military/logistics bases	Referenced by users, purchases, transfers, expenditures
equipment_types	Equipment categories/types and units	Referenced by assets, purchases, transfer_items, expenditures
assets	Individual serialized assets	FK → equipment_types, bases
purchases	Purchased quantities for a base	FK → bases, equipment_types, users
transfers	Header for movement between bases	FK → source/destination bases, users
transfer_items	Equipment and quantity inside a transfer	FK → transfers, equipment_types
assignments	Assignment of serialized assets to personnel	FK → assets, users
expenditures	Consumed/expended quantities	FK → bases, equipment_types, users
audit_logs	Immutable-style business audit trail	FK → users; entity reference

Inventory calculation

Net Movement = Purchases + Transfer In – Transfer Out.

Closing Physical Inventory = Opening Balance + Purchases + Transfer In – Transfer Out – Expenditure.

Assignments are tracked separately because assigning an asset to personnel normally changes its availability but does not necessarily mean the organization no longer owns it.

Transfer transaction

A transfer is executed as a database transaction: validate source inventory → create transfer header/items → apply inventory movement → create audit record → commit. If a validation or database operation fails, the transaction is rolled back to prevent partial movement records.

4. RBAC Explanation

Role	Access
Admin	Full access to users, bases, equipment, purchases, transfers, assignments, expenditures, dashboards and audit information
Base Commander	Access to operational data and permitted operations for the assigned base; backend checks prevent access to other bases
Logistics Officer	Limited operational access focused on purchases and transfers, subject to backend authorization.

Enforcement: Authentication identifies the user through a signed JWT. FastAPI dependencies/middleware check the role before protected operations. Base-scoped users are additionally checked against the base attached to the authenticated user. Frontend controls are only for usability; authorization is enforced server-side.

5. API Logging

The system distinguishes technical API/request logs from business audit logs. Request logging can record HTTP method, route, status code, response time and authenticated user. Business audit logging records meaningful state-changing actions such as PURCHASE_CREATED, TRANSFER_CREATED, ASSIGNMENT_CREATED and EXPENDITURE_CREATED.

An audit record contains fields such as:

- user_id
- action
- entity_type
- entity_id
- old_values / new_values (JSON)
- IP address
- timestamp

For sensitive production deployments, audit records should be protected from ordinary modification/deletion and retained according to the organization's audit and compliance policy.

6. Setup Instructions

Prerequisites

- Python 3.10+ recommended
- Node.js and npm
- Git
- A MySQL instance (local or managed cloud)
- Optional: MySQL Workbench or another SQL client

Backend

```
cd backend
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Create a backend .env file containing MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD and MYSQL_DATABASE. Do not commit .env or database credentials to source control.

Database

Create or select the MySQL database, then run the project's schema/migration scripts. For a cloud provider, use the provider's connection variables. For production deployment, prefer the provider's private network and secret/reference variables when the backend and database run in the same cloud project.

Frontend

```
cd frontend
npm install
npm run dev
```

Configure the frontend API base URL using an environment variable such as VITE_API_BASE_URL. The frontend should never contain database credentials.

Typical development flow

7. API Endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate user and issue JWT
GET	/api/v1/auth/me	Return authenticated user context
GET	/api/v1/dashboard/summary	Dashboard balances and metrics
GET	/api/v1/dashboard/movements	Movement breakdown
POST	/api/v1/purchases	Create purchase record
GET	/api/v1/purchases	List/filter purchases
POST	/api/v1/transfers	Create transfer between bases
GET	/api/v1/transfers	List transfer history
POST	/api/v1/assignments	Assign serialized asset
POST	/api/v1/expenditures	Record expenditure
GET	/api/v1/audit-logs	Authorized audit-log access

Example transfer request

```
{
  "from_base_id": 1,
  "to_base_id": 2,
  "items": [{"equipment_type_id": 2, "quantity": 20}]
}
```

FastAPI/OpenAPI can be used to interactively inspect and test the documented endpoints during development.

8. Login Credentials

Use the working credentials configured in the deployed/demo platform. For security, this documentation template does not invent or expose a password that has not been verified. Replace the placeholders below with the actual test credentials before submission.

Role	Email / Username	Password
Admin	YOUR_ADMIN_EMAIL	YOUR_ADMIN_PASSWORD
Base Commander	YOUR_COMMANDER_EMAIL	YOUR_COMMANDER_PASSWORD
Logistics Officer	YOUR_LOGISTICS_EMAIL	YOUR_LOGISTICS_PASSWORD

Security note: Use dedicated demo/test accounts rather than personal or production credentials. If the evaluator requires credentials in the PDF, include only accounts created specifically for evaluation.

Submission Checklist

- PDF attached with all eight requested sections.
- Demo URL included if the platform is deployed.
- Working test credentials inserted into Section 8.
- 3–5 minute walkthrough video submitted separately.
- Source repository URL included if requested.
- No database passwords, API keys or private secrets included in the PDF or repository.